

Progettazione di Applicazioni Mobili

Flutter Programming

Widgets – Input

Stefano Burigat

Dipartimento di Scienze Matematiche, Informatiche e
Fisiche - Università di Udine

stefano.burigat@uniud.it



Input: TextField

- TextField is the most commonly used text input widget
- You can add a label, icon, inline hint text, and error text by supplying an InputDecoration as the decoration property of the TextField
 - By default, a text field has a decoration that draws a divider below the text field
- If the user indicates that they are done typing in the field, the text field calls the onSubmitted callback

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext inContext) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: TextField(
            onSubmitted: (String value) {
              print(value);
            },
            decoration: InputDecoration(
              border: OutlineInputBorder(),
              labelText: 'YourInput',
            ), // InputDecoration
          ), // TextField, Center, Scaffold, /
        ), // Scaffold
      ), // MaterialApp
    ); // runApp
  }
}
```

Input: TextButton

- A TextButton is a text label displayed on a (zero elevation) Material widget that reacts to touches by filling with color
- TextButtons intentionally do not have visible borders and must therefore rely on their position relative to other content for context
- The onPressed property defines the callback that is called when the button is tapped or otherwise activated

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: TextButton(
            style: TextButton.styleFrom(
              foregroundColor: Colors.white,
              backgroundColor: Colors.blue,
              disabledForegroundColor: Colors.black12,
              disabledBackgroundColor: Colors.grey,
              padding: EdgeInsets.all(16),
            ),
            onPressed: () {
              print("Button pressed");
            },
            child: Text(
              "Text button",
              style: TextStyle(fontSize: 20),
            ), // Text
          ), // TextButton, Center, Scaffold, MaterialApp
        ), // Scaffold
      ), // MaterialApp
    ); // runApp
  }
}
```

Input: ElevatedButton

- An ElevatedButton is based on a Material widget whose elevation increases when the button is pressed
- If onPressed and onLongPress callbacks are null, then the button will be disabled and by default will resemble a TextButton in the disabledColor

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: TextButton(
            style: ElevatedButton.styleFrom(
              elevation: 12,
              foregroundColor: Colors.white,
              backgroundColor: Colors.blue,
              disabledForegroundColor: Colors.black12,
              disabledBackgroundColor: Colors.grey,
              padding: EdgeInsets.all(16),
            ),
            onPressed: () {
              print("Button pressed");
            },
            child: Text(
              "Text button",
              style: TextStyle(fontSize: 20),
            ), // Text
          ), // TextButton, Center, Scaffold, MaterialApp
        ), // Scaffold, MaterialApp
      ), // MaterialApp
    );
  }
}
```

Input: Checkbox

- To create a checkbox, use the Checkbox widget
- The Checkbox widget does not intrinsically have a text label
 - To achieve that, you'll need to build it yourself, usually by placing a Checkbox and a Text widget in a Row container
- Note that you need to deal with state to make the checkbox changes visible

```
class _MyApp extends State {  
  var _checkboxValue = false;  
  
  @override  
  Widget build(BuildContext inContext) {  
    return MaterialApp(home: Scaffold(  
      body: Center(  
        child: Checkbox(  
          value: _checkboxValue,  
          onChanged: (bool inValue) {  
            setState(() {  
              _checkboxValue = inValue;  
            });  
          })  
        ), // Checkbox  
      )); // Center, Scaffold, MaterialApp  
    }  
  }  
}
```

Input: Switch

- To create a switch, use the Switch widget
- It works exactly like the Checkbox widget

```
class _MyApp extends State {  
  var _switchValue = false;  
  
  @override  
  Widget build(BuildContext inContext) {  
    return MaterialApp(home: Scaffold(  
      body: Center(  
        child: Switch(  
          value: _switchValue,  
          onChanged: (bool inValue) {  
            setState(() {  
              _switchValue = inValue;  
            });  
          },  
        ), // Switch  
      )), // Center, Scaffold, MaterialApp  
    );  
  }  
}
```

Input: Slider

- A Slider is a widget that is a line and a little handle, called a thumb, that the user drags to select a value from a predefined range
- The important properties are min and max, which define the lower and upper limits of the range of values the user can choose from, and value, which is its current value

```
class _MyApp extends State {  
  var _sliderValue = .3;  
  
  @override  
  Widget build(BuildContext inContext) {  
    return MaterialApp(home: Scaffold(  
      body: Center(  
        child: Slider(  
          min : 0, max : 20,  
          value : _sliderValue,  
          onChanged : (inValue) {  
            setState(() => _sliderValue = inValue);  
          }  
        ), // Slider  
      )), // Center, Scaffold, MaterialApp  
    )  
  }  
}
```

Input: Radio

- The Radio widget is a lot like CheckBox or Switch except that it never exists on its own
- A Radio widget always has one or more sister Radio widgets, and they are mutually exclusive: selecting any one Radio causes the other in its group to be de-selected
 - by virtue of them all having the same groupValue property value, they become part of the same group

```
class _MyApp extends State {  
  var _radioValue = 1;  
  
  @override  
  Widget build(BuildContext inContext) {  
    return MaterialApp(  
      home: Scaffold(  
        body: Column(children: [  
          Row(children: [  
            Radio(  
              value: 1,  
              groupValue: _radioValue,  
              onChanged: (int inValue) {  
                setState(() {  
                  _radioValue = inValue;  
                });  
              }), // Radio  
            Text("Option 1")  
          ]), // Row  
          Row(children: [  
            Radio(  
              value: 2,  
              groupValue: _radioValue,  
              onChanged: (int inValue) {  
                setState(() {  
                  _radioValue = inValue;  
                });  
              }), // Radio  
            Text("Option 2")  
          ]), // Row  
        ])); // Column, Scaffold, MaterialApp  
      }  
    )  
  }  
}
```

Input: Date and Time

- Flutter provides two functions, `showDatePicker()` and `showTimePicker()`, for you to call to show UI components for choosing a date or time in an app
 - `showDatePicker()` requires the build context, the `initialDate` that's selected by default, and the `firstDate` and `lastDate` that the picker will allow to be chosen
 - for `showTimePicker()`, only the build context and the `initialTime` are necessary
- Both of these functions are asynchronous, so we need some async functions to make use of them, namely, the `_selectDate()` and `_selectTime()` methods, which are called from the two buttons in the main layout

Input: Date and Time

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(home: Scaffold(body: Home()));
  }
}

class Home extends StatelessWidget {
  Future<void> _selectDate(inContext) async {
    DateTime selectedDate = await showDatePicker(
      context: inContext,
      initialDate: DateTime.now(),
      firstDate: DateTime(2017),
      lastDate: DateTime(2021));
    print(selectedDate);
  }

  Future<void> _selectTime(inContext) async {
    TimeOfDay selectedTime = await showTimePicker(
      context: inContext,
      initialTime: TimeOfDay.now(),
    );
    print(selectedTime);
  }
}
```

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Column(
      children: [
        Container(
          height: 50,
        ), // Container
        ElevatedButton(
          child: Text("Show DatePicker"),
          onPressed: (){
            _selectDate(context);
          },
        ), // ElevatedButton
        ElevatedButton(
          child: Text("Show TimePicker"),
          onPressed: (){
            _selectTime(context);
          },
        ), // ElevatedButton
      ],
    ), // Column
  ); // Scaffold
}
```

Input: Dismissible

- The Dismissible widget is an element that the user can get rid of by flinging it in a given direction
- The widget has a direction property that specifies which direction it can be dragged in
- A key also must be defined for this to work; in this example, a GlobalKey instance fulfills this requirement

```
class MyApp extends StatefulWidget {  
  MyApp({Key key}) : super(key: key);  
  
  @override  
  _MyApp createState() => _MyApp();  
}  
  
class _MyApp extends State {  
  var _switchValue = false;  
  
  @override  
  Widget build(BuildContext inContext) {  
    return MaterialApp(  
      home: Scaffold(  
        body: Center(  
          child: Dismissible(  
            key: GlobalKey(),  
            direction: DismissDirection.vertical,  
            child: Container(  
              color: Colors.yellow,  
              width: 100,  
              height: 50,  
              child: Text("Swipe me"))), // Contain  
          )); // Center, Scaffold, MaterialApp  
        }  
      }  
    )
```

Input: Form

- The Form widget is used frequently around user input
 - it provides you some common functionality including saving the form data, resetting it and validating it
- Form is a container for form fields: there is a FormField widget that wraps all input fields and which are then made children of the Form widget

Input: Form

```
import "package:flutter/material.dart";

void main() => runApp(MyApp());

class MyApp extends StatefulWidget {
  MyApp({Key key}) : super(key: key);

  @override
  _MyApp createState() => _MyApp();
}

class LoginData {
  String username = "";
  String password = "";
}

class _MyApp extends State {
  LoginData _loginData = new LoginData();
  GlobalKey<FormState> _formKey = new GlobalKey<FormState>();

  @override
  Widget build(BuildContext inContext) {
    return MaterialApp(
      home: Scaffold(
        body: Container(
          padding: EdgeInsets.all(50.0),
          child: Form(
            key: this._formKey,
            child: Column(children: [

```

```
TextFormField(
  keyboardType: TextInputType.emailAddress,
  validator: (String inValue) {
    if (inValue.length == 0) {
      return "Please enter username";
    }
    return null;
  },
  onSave: (String inValue) {
    this._loginData.username = inValue;
  },
  decoration: InputDecoration(
    hintText: "none@none.com",
    labelText: "Username (eMail address)")), // Inp
TextFormField(
  obscureText: true,
  validator: (String inValue) {
    if (inValue.length < 10) {
      return "Password must be >=10 in length";
    }
    return null;
  },
  onSave: (String inValue) {
    this._loginData.password = inValue;
  },
  decoration: InputDecoration(
    hintText: "Password", labelText: "Password")), // P
ElevatedButton(
  child: Text("Log In!"),
  onPressed: () {
    if (_formKey.currentState.validate()) {
      _formKey.currentState.save();
      print("Username: ${_loginData.username}");
      print("Password: ${_loginData.password}");
    }
  }) // RaisedButton
))))); // Column, Form, Container, Scaffold, MaterialApp
```

Input: Form

- The LoginData class consolidates all the user input (username and password) in one object, making it easier to work with
- GlobalKey is a key that is unique across the entire app
 - This usually comes into play as the value of the key property of a widget, which determines how a widget replaces another in the widget tree
 - you can access any property of any widget or execute methods directly on a widget, as long as it has a key

Input: Form

- In the usual build() method we have a Form widget which has a key property and a child property
- The child is a container (a Column) with three children: the username entry field, password entry field, and the Log In button
 - The username and password fields use the TextFormField widget, a widget that effectively combines a FormField widget, which wraps all fields in a Form, and a TextField widget for getting user text input
 - The keyboardType property allows us to display the most appropriate keyboard based on the field type
 - The validator property defines a function that will perform validation on the field when the Log In button is clicked

Input: Form

- The `onSaved` property specifies a callback function that will fire when the `save()` method of the Form is called, which will happen when the user presses the Button in the form
 - The `onSaved` handler function just stores the field value into the appropriate `_loginData` variable's field
- The `decoration` property is an instance of `InputDecoration` and is commonly used to declare some `hintText` (shown in the field when nothing has yet been entered) and a label for the field via the `labelText` property, which gets shown above the field
- The `obscureText` property of the password field allows to hide what the user enters in the field

Input: Form

- In the button code, we call the `validate()` method through the `_formKey` variable which provides us a reference to the form widget
 - within that widget is the `currentState` property that contains the values currently entered on the form
 - since each field has a validator function attached, `validate()` knows how to call each of them and either display the error fields or else return true if the form is valid according to the validators on all its fields
 - In that case, we call `save()` on the `currentState`, which results in all the `onSaved` handlers firing and thus the form data is stored in `_loginData`